

Entity Framework Core (EF Core)

Introduction

Entity Framework Core (EF Core) is Microsoft's modern, lightweight, open-source Object-Relational Mapper (ORM) for .NET applications. It enables developers to interact with relational databases using .NET objects instead of writing raw SQL queries for every database operation.

EF Core supports multiple database providers, including:

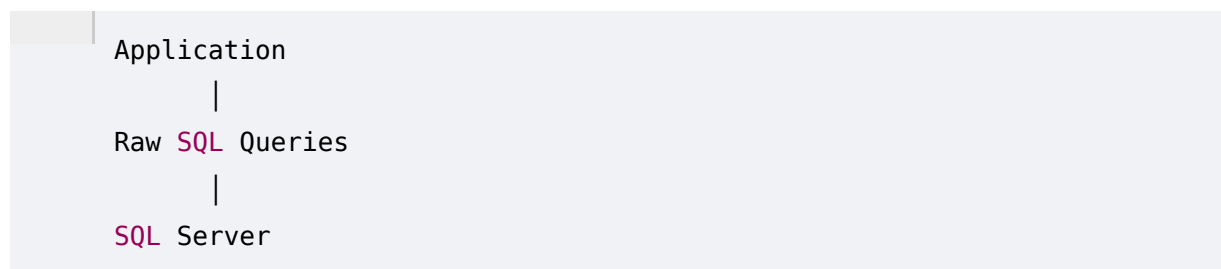
- Microsoft SQL Server
- PostgreSQL
- MySQL
- SQLite
- Oracle
- Azure Cosmos DB (NoSQL)
- In-Memory Database (for testing)

Entity Framework Core is the recommended data access technology for modern ASP.NET Core applications. It integrates seamlessly with Dependency Injection, LINQ, ASP.NET Core, and Minimal APIs.

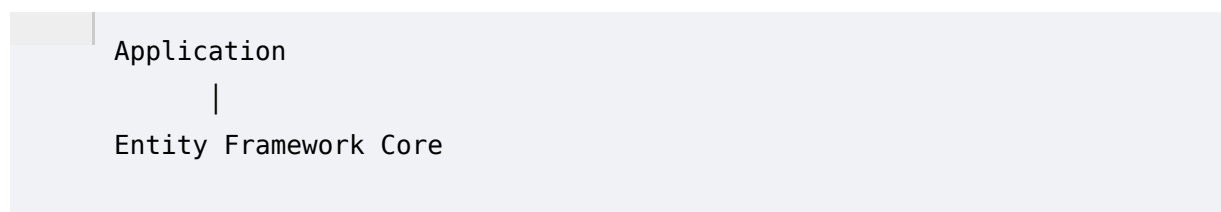
What is an ORM?

An **Object-Relational Mapper (ORM)** bridges the gap between object-oriented programming and relational databases.

Without an ORM:



With EF Core:



|
SQL Server

Developers work with C# objects while EF Core automatically translates operations into SQL statements.

Why Use Entity Framework Core?

Traditional database development requires writing SQL for every operation.

Example using ADO.NET:

```
SqlConnection connection = new SqlConnection(connectionString);

SqlCommand command = new SqlCommand(
    "SELECT * FROM Products",
    connection);

connection.Open();

SqlDataReader reader = command.ExecuteReader();
```

With EF Core:

```
var products = dbContext.Products.ToList();
```

Benefits include:

- Less code
- Automatic SQL generation
- Strong typing
- LINQ support
- Change tracking
- Built-in migrations
- Better maintainability

EF Core Architecture

```
Application
|
Repository / Service
|
```

```
DbContext
|
Entity Framework Core
|
Database Provider
|
SQL Server
```

The `DbContext` acts as the primary bridge between the application and the database.

Key Components

Entity Framework Core consists of several important components.

Entity

Represents a database table.

```
public class Product
{
    public int Id { get; set; }

    public string Name { get; set; }

    public decimal Price { get; set; }
}
```

DbContext

Represents a database session.

```
public class AppDbContext : DbContext
{
    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
}
```

DbSet

Represents a table.

```
public DbSet<Product> Products { get; set; }
```

Each `DbSet` corresponds to one database table.

Configuring EF Core

Install packages.

```
dotnet add package Microsoft.EntityFrameworkCore

dotnet add package Microsoft.EntityFrameworkCore.SqlServer

dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Register DbContext.

```
builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

Connection string.

```
{
    "ConnectionStrings": {
        "DefaultConnection":

        "Server=.;Database=StoreDB;Trusted_Connection=True;TrustServerCertificate=True;"
    }
}
```

CRUD Operations

Insert

```
var product = new Product
{
    Name = "Laptop",
```

```
        Price = 75000
    };

    db.Products.Add(product);

    db.SaveChanges();
```

Generated SQL

```
INSERT INTO Products
(Name, Price)
VALUES
('Laptop', 75000)
```

Read

Get all records.

```
var products = db.Products.ToList();
```

Get single record.

```
var product = db.Products.Find(1);
```

Filter records.

```
var expensiveProducts =
    db.Products
        .Where(p => p.Price > 50000)
        .ToList();
```

Update

```
var product = db.Products.Find(1);

product.Price = 80000;

db.SaveChanges();
```

Generated SQL

```
UPDATE Products
SET Price = 80000
```

```
WHERE Id = 1
```

Delete

```
var product = db.Products.Find(1);

db.Products.Remove(product);

db.SaveChanges();
```

Generated SQL

```
DELETE FROM Products
WHERE Id = 1
```

LINQ with EF Core

Language Integrated Query (LINQ) is one of EF Core's most powerful features.

Example:

```
var products = db.Products
    .Where(p => p.Price > 5000)
    .OrderBy(p => p.Name)
    .ToList();
```

Equivalent SQL:

```
SELECT *
FROM Products
WHERE Price > 5000
ORDER BY Name
```

Common LINQ methods:

- Where()
- Select()
- OrderBy()
- OrderByDescending()
- First()
- FirstOrDefault()
- Single()

- Count()
 - Sum()
 - Average()
 - Max()
 - Min()
 - Any()
 - All()
-

Asynchronous Operations

Use asynchronous methods for database access.

```
var products =  
    await db.Products.ToListAsync();
```

Insert:

```
await db.Products.AddAsync(product);  
  
await db.SaveChangesAsync();
```

Advantages:

- Better scalability
 - Non-blocking I/O
 - Improved application performance
-

Migrations

Migrations manage database schema changes.

Create migration.

```
dotnet ef migrations add InitialCreate
```

Apply migration.

```
dotnet ef database update
```

Generated migration example:

```
migrationBuilder.CreateTable(  
    name: "Products",
```

```
columns: table => new
{
    Id = table.Column<int>(),
    Name = table.Column<string>(),
    Price = table.Column<decimal>()
};
```

Benefits:

- Version-controlled schema
- Easy deployment
- Automatic updates

Code First Approach

In Code First, developers create C# classes first.

```
Entity Classes
|
Migration
|
Database
```

Advantages:

- Faster development
- Strong typing
- Easier maintenance

Suitable for new applications.

Database First Approach

In Database First, an existing database is used.

```
Existing Database
|
Scaffold Command
|
Entity Classes
```

Scaffold example:

```
dotnet ef dbcontext scaffold
```

Suitable for:

- Legacy systems
- Existing enterprise databases

Relationships

One-to-Many

```
graph TD; Category --> Products; Category --- Many; style Many fill:none,stroke:none; style Products fill:none,stroke:none;
```

Category
|
|
Many
Products

Example:

```
public class Category
{
    public int Id { get; set; }

    public ICollection<Product> Products { get; set; }
}

public class Product
{
    public int Id { get; set; }

    public int CategoryId { get; set; }

    public Category Category { get; set; }
}
```

One-to-One

```
graph TD; Employee --> EmployeeDetail; Employee --- EmployeeDetail; style Employee fill:none,stroke:none; style EmployeeDetail fill:none,stroke:none;
```

Employee
|
EmployeeDetail

Many-to-Many

```
Students
  |
Enrollment
  |
Courses
```

EF Core automatically manages many-to-many relationships without requiring an explicit join entity in many scenarios.

Loading Related Data

Eager Loading

```
var products =
    db.Products
        .Include(p => p.Category)
        .ToList();
```

Loads related data immediately.

Explicit Loading

```
db.Entry(product)
    .Reference(p => p.Category)
    .Load();
```

Loads related data only when requested.

Lazy Loading

Related entities are loaded automatically when accessed.

Requires Lazy Loading proxies.

Change Tracking

EF Core automatically tracks changes made to entities.

```
var product = db.Products.Find(1);

product.Price = 60000;

db.SaveChanges();
```

EF Core detects the modified property and generates the appropriate SQL update statement.

Transactions

EF Core supports database transactions.

```
using var transaction =
    db.Database.BeginTransaction();

try
{
    db.SaveChanges();

    transaction.Commit();
}
catch
{
    transaction.Rollback();
}
```

Transactions ensure data consistency.

Raw SQL

Although EF Core generates SQL automatically, raw SQL can also be executed.

```
var products =
    db.Products
        .FromSqlRaw(
            "SELECT * FROM Products")
        .ToList();
```

Execute stored procedure.

```
db.Products
    .FromSqlRaw("EXEC GetProducts")
    .ToList();
```

Use raw SQL only when necessary.

Concurrency

EF Core supports optimistic concurrency.

Example:

```
public byte[] RowVersion { get; set; }
```

Decorate with:

```
[Timestamp]  
public byte[] RowVersion { get; set; }
```

This helps prevent overwriting changes made by another user.

Performance Optimization

Best practices:

- Use asynchronous methods.
- Retrieve only required columns.
- Use `AsNoTracking()` for read-only queries.
- Avoid unnecessary `Include()` statements.
- Use pagination.
- Create indexes in the database.
- Minimize database round trips.
- Avoid loading large object graphs unnecessarily.

Example:

```
var products =  
    db.Products  
        .AsNoTracking()  
        .ToList();
```

Dependency Injection

Register DbContext.

```
builder.Services.AddDbContext<AppDbContext>();
```

Inject into repository.

```
public class ProductRepository  
{
```

```
private readonly AppDbContext _db;

public ProductRepository(AppDbContext db)
{
    _db = db;
}
}
```

ASP.NET Core automatically provides the DbContext instance.

Advantages of EF Core

- Reduces boilerplate code
 - Automatic SQL generation
 - Strong typing
 - LINQ support
 - Built-in migrations
 - Cross-platform
 - Dependency Injection integration
 - Multiple database providers
 - Automatic change tracking
 - High developer productivity
-

Limitations

Although EF Core is powerful, it is not always the best solution.

Potential limitations include:

- Complex SQL queries may require raw SQL.
- Slight performance overhead compared to optimized ADO.NET.
- Understanding LINQ translation is important.
- Poorly designed queries can impact performance.

For performance-critical scenarios, developers often combine EF Core with ADO.NET or stored procedures.

Best Practices

- Use Dependency Injection for DbContext.

- Prefer asynchronous methods (`ToListAsync` , `SaveChangesAsync`).
- Keep DbContext short-lived (Scoped lifetime).
- Use DTOs instead of exposing entities directly.
- Apply migrations through source control.
- Use `AsNoTracking()` for read-only queries.
- Avoid business logic inside DbContext.
- Handle exceptions gracefully.
- Profile generated SQL for performance tuning.
- Follow the Repository or Service pattern where appropriate.

Entity Framework Core Request Flow



This layered architecture separates business logic from data access and promotes maintainability and testability.

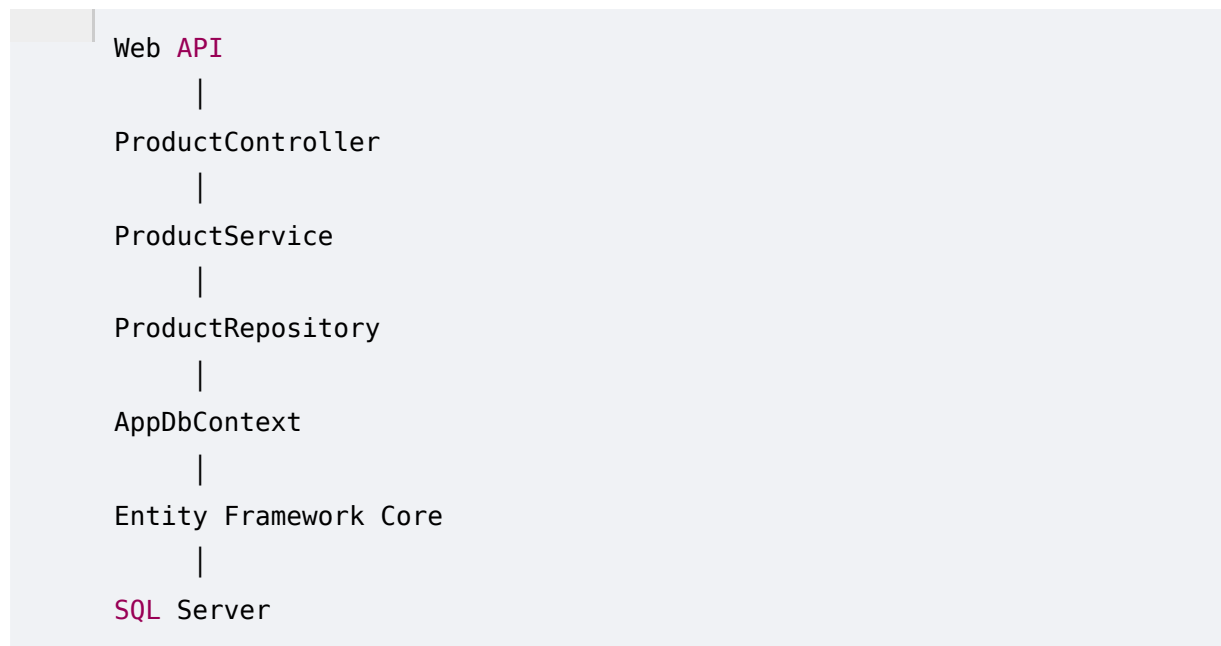
Entity Framework Core vs ADO.NET

Feature	Entity Framework Core	ADO.NET

	rk Core	
SQL Writing	Mostly Automatic	Manual
Performance	Very Good	Excellent
Learning Curve	Easier	Moderate
LINQ Support	Yes	No
Change Tracking	Yes	No
Migrations	Yes	No
CRUD Development Speed	Fast	Slower
Fine-Grained SQL Control	Moderate	Excellent
Best Use Case	Business Applications	High-Performance Data Access

Real-World Example

Consider an e-commerce application.



When a client requests `GET /api/products`, the controller calls the service, which retrieves data through the repository. The repository uses `AppDbContext` and LINQ to query SQL Server. EF Core translates the LINQ expression into SQL, executes it, maps the results back to C# objects, and returns them to the client. This abstraction simplifies development while maintaining a clean, layered architecture.

Summary

Entity Framework Core is Microsoft's modern Object-Relational Mapper (ORM) for .NET applications, providing a productive and maintainable way to interact with relational databases. By allowing developers to work with strongly typed C# objects instead of manually writing SQL for common operations, EF Core significantly reduces development effort while supporting advanced capabilities such as LINQ queries, change tracking, migrations, relationship management, asynchronous operations, and multiple database providers. When used with proper architectural practices, Dependency Injection, and performance optimization techniques, Entity Framework Core becomes a powerful foundation for building scalable, enterprise-grade ASP.NET Core applications.